# self-signed certificate with openssl

Shinyoung You

# Generate certificate (cert.pem) and private key (key.pem)

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -sha256 -days 365 -nodes \
  -subj "/C=CA/ST=British Columbia/L=Vancouver/O=MyCompany/CN=localhost"
```

| Option | Value | Meaning |
|---|---|---|
| `openssl req` | - | Enter OpenSSL's certificate request mode |
| `-x509` | - | Skip CSR, directly create a self-signed certificate |
| `-newkey` | `rsa:4096` | Generate a new private key using RSA 4096-bit |
| `-keyout` | `key.pem` | Save the private key to this file |
| `-out` | `cert.pem` | Save the certificate to this file |

# Generate certificate (cert.pem) and private key (key.pem) (cont.)

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -sha256 -days 365 -nodes \
  -subj "/C=CA/ST=British Columbia/L=Vancouver/O=MyCompany/CN=localhost"
```

| Option | Value | Meaning |
|---|---|---|
| `-sha256` | - | Use SHA-256 algorithm for signing |
| `-days` | `365` | Certificate validity period (1 year) |
| `-nodes` | - | No passphrase on the private key |
| `-subj` | - | Set certificate identity info without interactive prompts |

# Generate certificate (cert.pem) and private key (key.pem) (cont.)

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -sha256 -days 365 -nodes \
 -subj "/C=CA/ST=British Columbia/L=Vancouver/O=MyCompany/CN=localhost"
```

| Option | Value | Meaning |
|---|---|---|
| `C` | `CA` | Country code (Canada) |
| `ST` | `British Columbia` | State / Province |
| `L` | `Vancouver` | Locality / City |
| `O` | `MyCompany` | Organization name |
| `CN` | `localhost` | Common Name — the domain this cert is valid for |

# node.js server

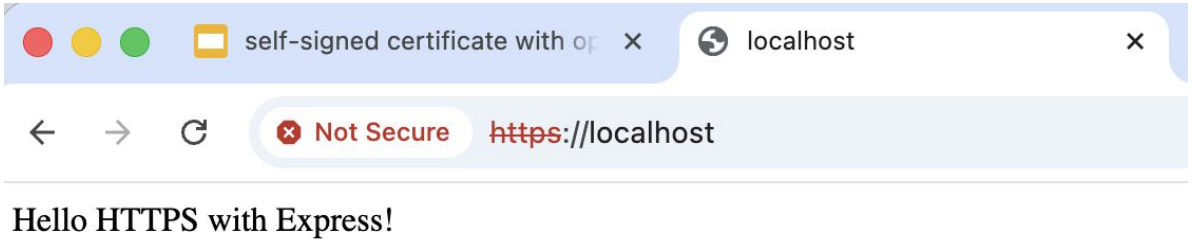
```
const https = require("https");
const fs = require("fs");
const express = require("express");

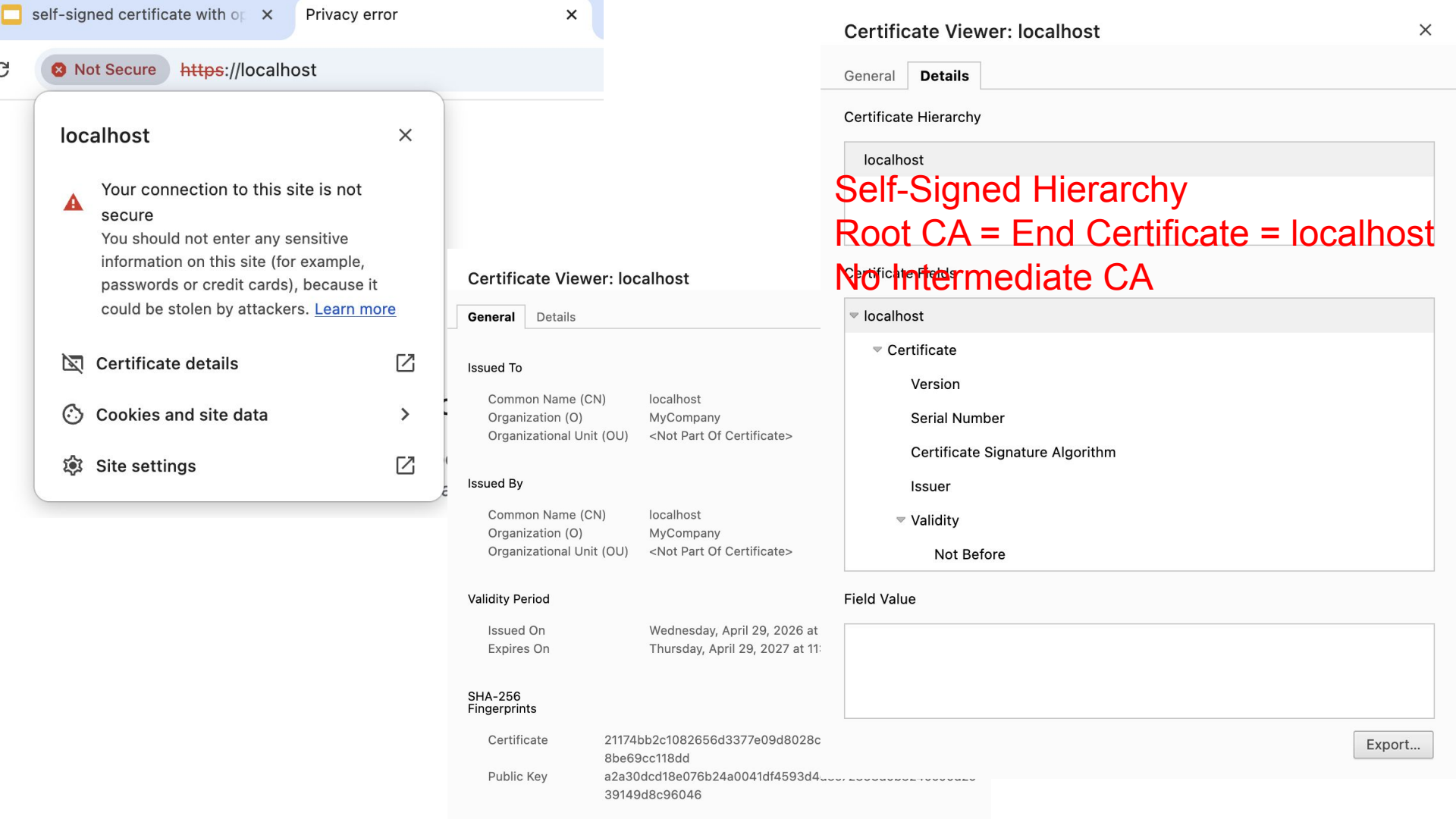
const app = express();

const options = {
  key: fs.readFileSync("key.pem"), // private key
  cert: fs.readFileSync("cert.pem"), // certificate
};

app.get("/", (req, res) => { // GET https://localhost/
  res.send("Hello HTTPS with Express!");
});

https.createServer(options, app).listen(443, () => {
  console.log("HTTPS server running on https://localhost");
});
```

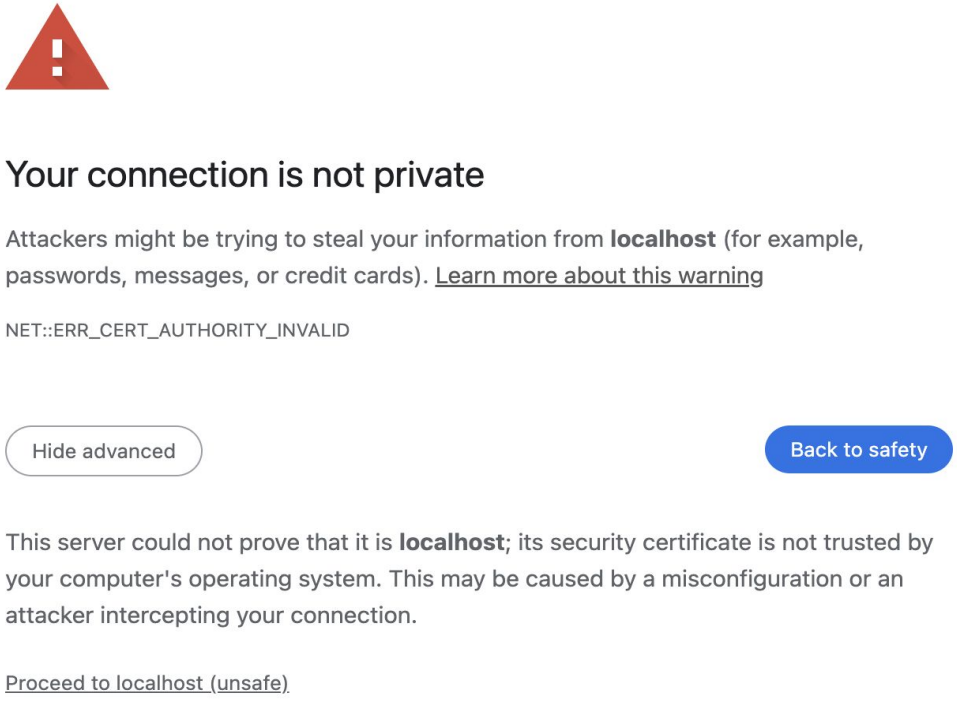



self-signed certificate with o
Privacy error
Not Secure
https://localhost
localhost
Your connection to this site is not secure
You should not enter any sensitive information on this site (for example, passwords or credit cards), because it could be stolen by attackers. Learn more
Certificate details
Cookies and site data
Site settings
Certificate Viewer: localhost
General
Details
Issued To
Common Name (CN) localhost
Organization (O) MyCompany
Organizational Unit (OU) <Not Part Of Certificate>
Issued By
Common Name (CN) localhost
Organization (O) MyCompany
Organizational Unit (OU) <Not Part Of Certificate>
Validity Period
Issued On Wednesday, April 29, 2026 at
Expires On Thursday, April 29, 2027 at 11
SHA-256 Fingerprints
Certificate 21174bb2c1082656d3377e09d8028c8be69cc118dd
Public Key a2a30dcd18e076b24a0041df4593d4
39149d8c96046
Certificate Viewer: localhost
General
Details
Certificate Hierarchy
localhost
Self-Signed Hierarchy
Root CA = End Certificate = localhost
No Intermediate CA
Certificate Fields
localhost
Certificate
Version
Serial Number
Certificate Signature Algorithm
Issuer
Validity
Not Before
Field Value
Export...

# It’s https, why am I getting warning?

ERR_CERT_AUTHORITY_INVALID:

it should be signed by a trusted CA (Certificate Authority)



# To remove the warning, use mkcert command

home@MacBook-Pro m4 % mkcert -install # mkcert registers your Mac as a local CA (fixes the trusted CA problem)
The local CA is already installed in the system trust store! 👍
The local CA is now installed in the Firefox trust store (requires browser restart)! 🦊
The local CA is now installed in Java's trust store! ☕

home@MacBook-Pro m4 % mkcert localhost # mkcert automatically generates a cert with SAN included

Created a new certificate valid for the following names 📜
 - "localhost"

The certificate is at "./localhost.pem" and the key at "./localhost-key.pem" ✅

It will expire on 29 July 2028 🗓️

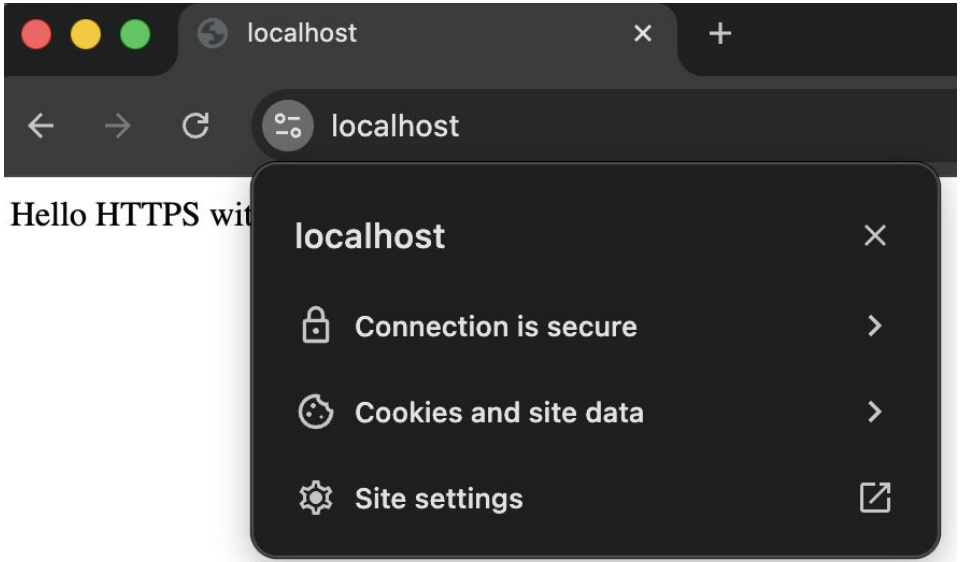
home@MacBook-Pro m4 % ls

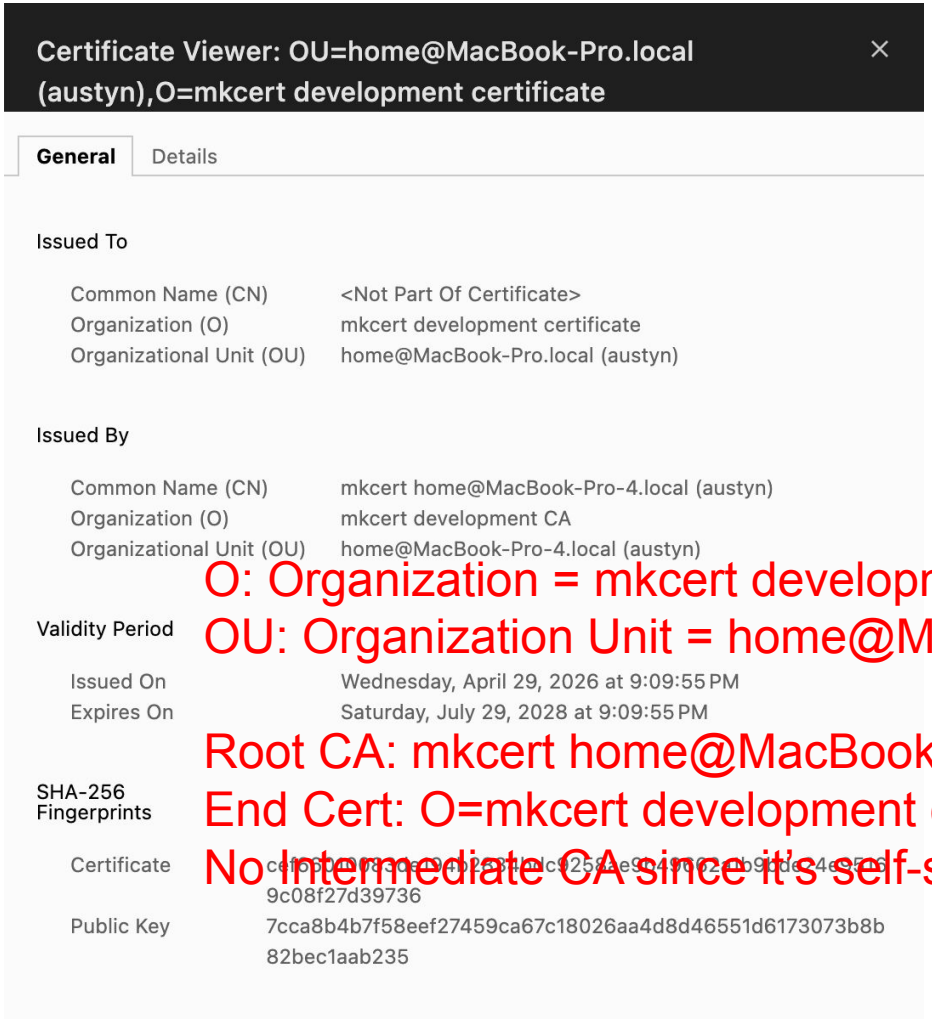
| | | |
|---|---|---|
| cert.pem | localhost-key.pem | package.json |
| detail | localhost.pem | server.js |
| imgs | node_modules | |
| key.pem | package-lock.json | |

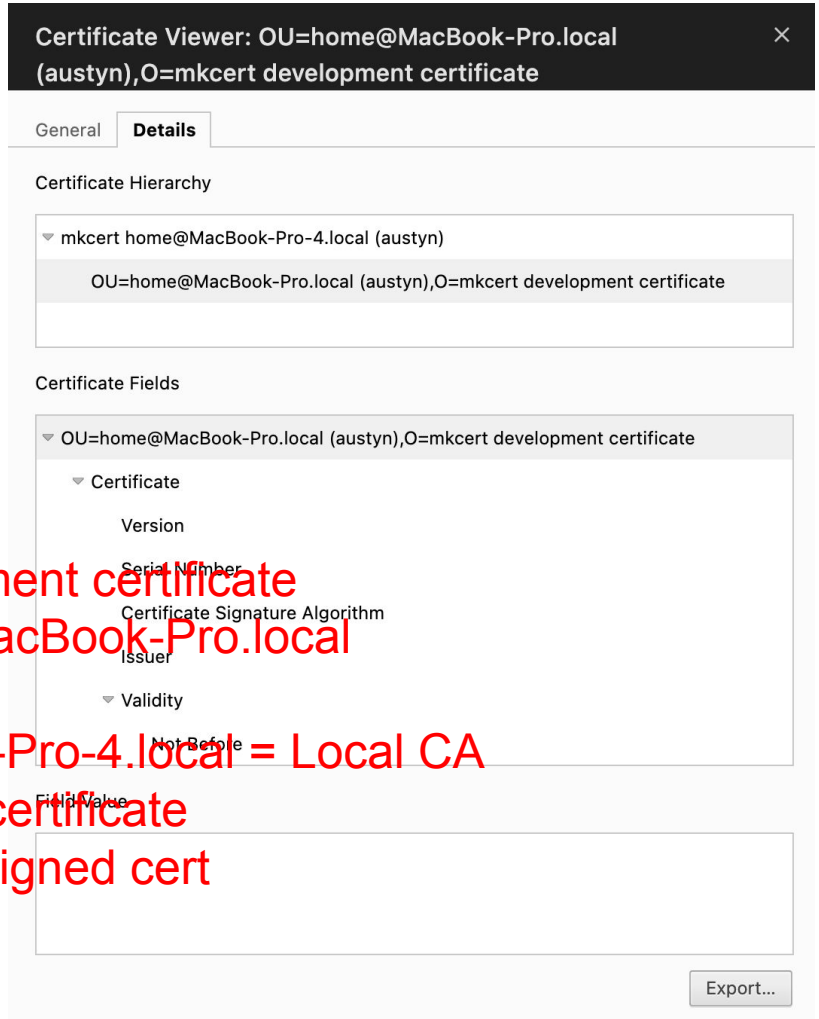
# Change the filenames in server code

"key.pem" → "localhost-key.pem"
"cert.pem" → "localhost.pem"

```
const options = {
   key: fs.readFileSync("localhost-key.pem"), // private key
   cert: fs.readFileSync("localhost.pem"), //certificate
};
```






O: Organization = mkcert development certificate

OU: Organization Unit = home@MacBook-Pro.local

Root CA: mkcert home@MacBook-Pro-4.local = Local CA

End Cert: O=mkcert development certificate

No Intermediate CA since it's self-signed cert

# SAN (Subject Alternative Name)

latest chrome browser doesn’t trust certificate without SAN.